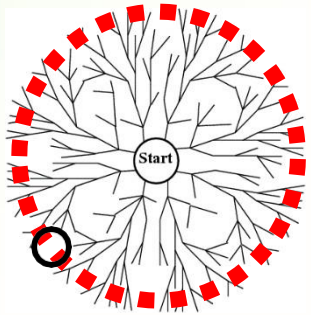
Advanced Search (Optimization)

Search vs. Optimization



Before, the goal was simply to find a path from the start state to the goal state using search strategies such as uninformed methods or informed approaches like A*.

New setting: **Optimization**

- States s have values $f(s)$

Goal: Find s with optimal value $f(s)$

- Sometimes we **maximize** $f(s)$(e.g., satisfaction, profit)
- Sometimes we **minimize** $f(s)$(e.g., cost, distance)

Challenge: Too many states for uninformed and informed approaches

- Cannot enumerate all paths
- Function may not be differentiable (cannot use gradient descent)

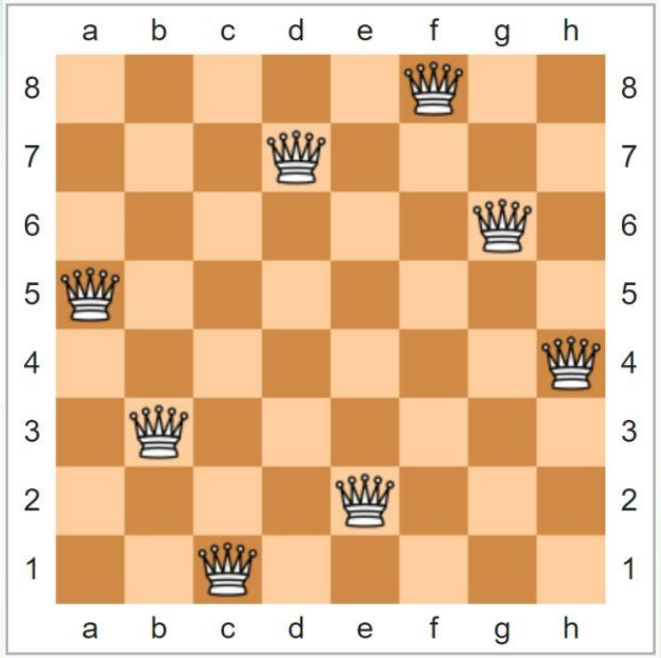
n-Queens

Classic puzzle: Place n queens on an $n \times n$ chessboard so that no two queens attack each other (same row, column, or diagonal)

State: Configuration of the board (position of all queens)

Objective function $f(s)$: Number of non-attacking queen pairs

Goal: Maximize $f(s)$. Ideally, with all combinations: $f(s) = \frac{n(n-1)}{2}$.



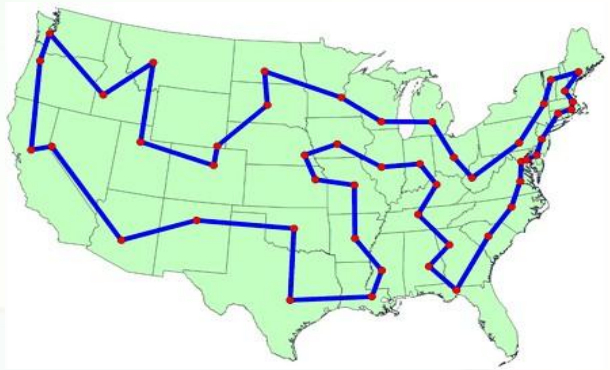
Why optimization instead of search?

This problem can be solved using traditional search methods. However, consider when n, the number of queens and number of sides on the chessboard, becomes large.

For large n (e.g., $n = 1000$), the search space becomes intractable; optimization methods will scale much better

nano banana

Traveling Salesperson Problem (TSP)



Problem: Given a complete, weighted graph $G = (V, E)$, find a *Hamiltonian cycle* that minimizes total distance

- Visits each node exactly once
- Returns to the initial node
- Minimizes total distance

State: A particular cyclic path (ordered list of nodes, e.g., A-B-C-D-E-A)
Objective function $\boldsymbol{f}(s)$: Total weight of the path (e.g., total miles)
Goal: Minimize $\boldsymbol{f}(s)$

Why optimization instead of search?

Number of possible paths in a complete weighted graph where every permutation of cities gives a valid tour : $\frac{(n-1)!}{2}$.

For $n = 10$ cities, there are ~181,000 tours; $n = 20$ → ~60 quintillion.

- Even with good heuristics (e.g., MST), A* does not scale due to exponential state space and memory requirements
- We only need the best tour, not the path to find it

Boolean Satisfiability

Problem: Given a Boolean formula in CNF, find a satisfying assignment

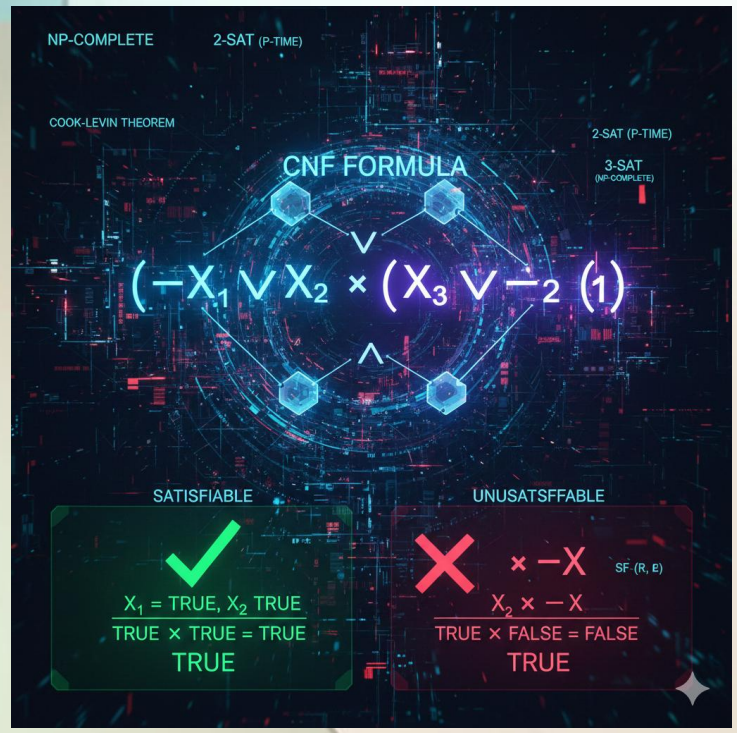
State: Assignment to variables (e.g., A=T， B=F， C=T， D=T， E=T)

Objective function $f(s)$: Number of satisfied clauses

Goal: Maximize $f(s)$ (MAX-SAT: satisfy the maximum number of clauses)

Example

(A ∨ ¬B ∨ C) ∧
(¬A ∨ C ∨ D) ∧
(B ∨ D ∨ ¬E) ∧
(¬C ∨ ¬D ∨ ¬E) ∧
(¬A ∨ ¬C ∨ E)



Why optimization instead of search?

2^n possible assignments of true / false.

For hard or unsatisfiable instances, search may fail to quickly find a solution, but optimization techniques can still produce good assignments by maximizing satisfied clauses even when a perfect solution does not exist.

Hill Climbing

Basic algorithm:

- Start at one state,
- Move to a neighbor with a better $\boldsymbol{f}(s)$ value,
- Repeat until no neighbors have better $\boldsymbol{f}(s)$ value.

Critical Question: How do we define **neighbor**?

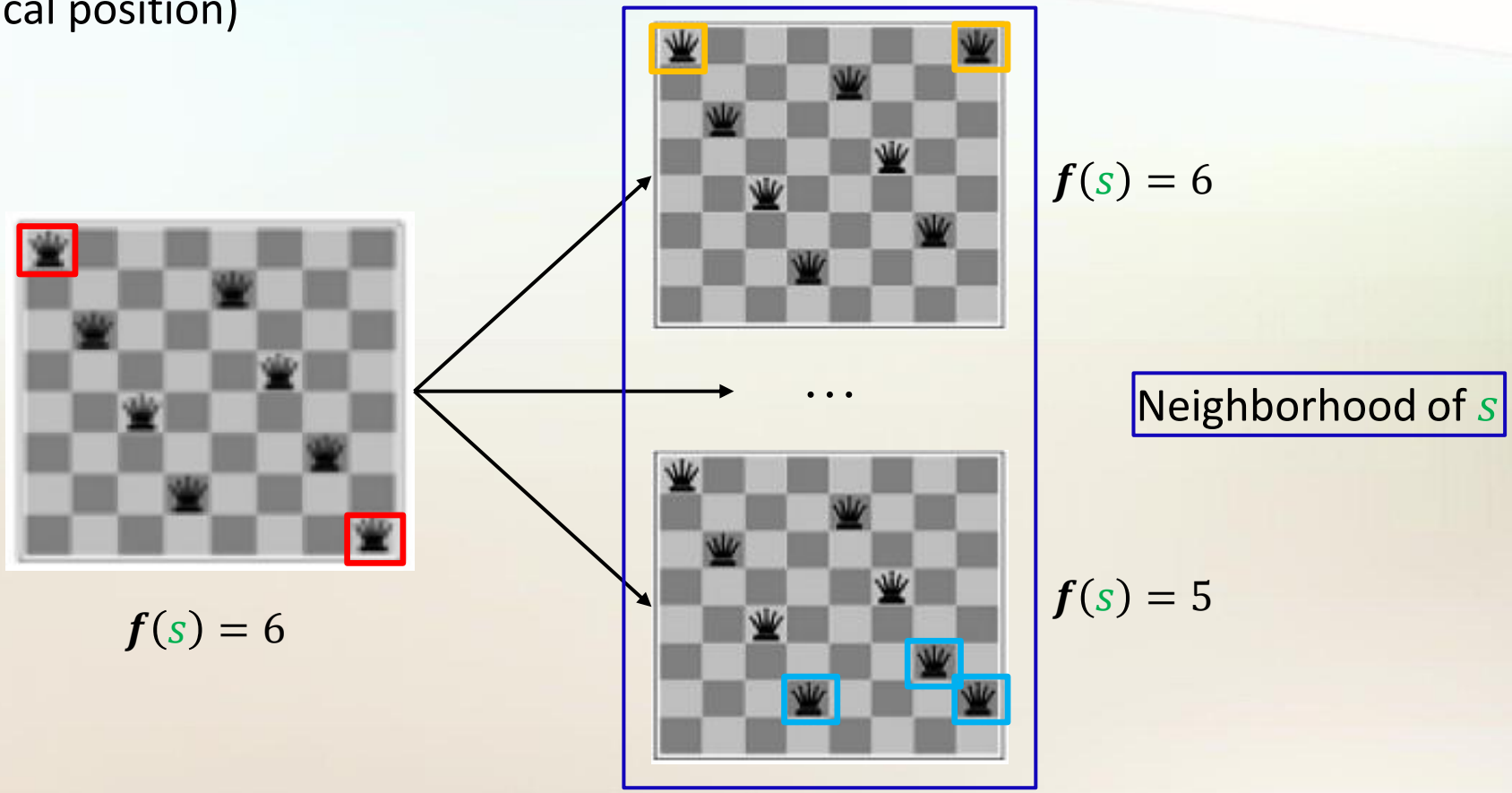
- **Neighbors** refer to states reachable by a single action from current state
- In optimization, we have **more freedom** in defining actions than in path-finding
  - Path-finding: actions often dictated by problem structure (roads, legal moves)
  - Optimization: we choose what counts as an action (which queen to move? how to modify tour?)
- This design choice significantly impacts algorithm performance
- Requires careful consideration of tradeoffs

Defining Neighbors: n-Queens

Approach: Small changes to current state

- Look at the most-conflicting column (if a tie, choose the rightmost column)
- Move queen in that column vertically to a different location

Result: Neighborhood contains $n - 1$ states
(one for each possible vertical position)

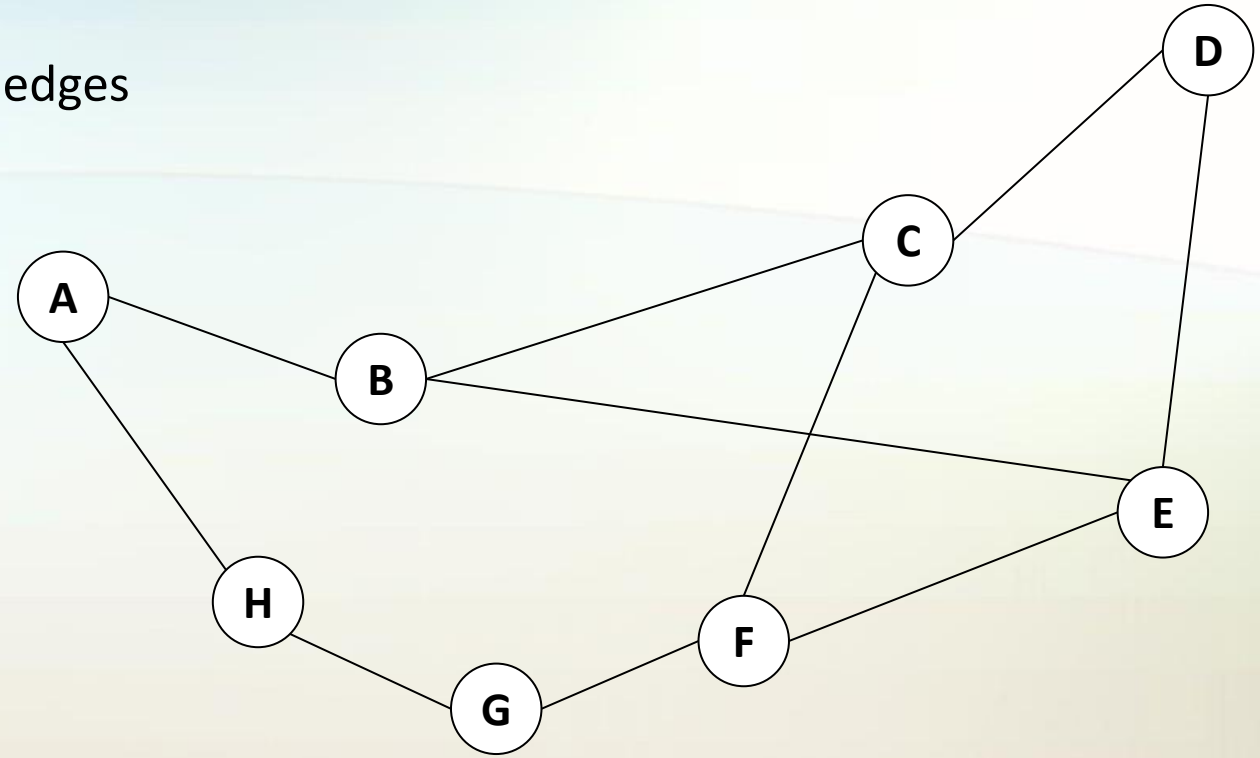


Objective function $f(s)$:
Number of non-attacking queen pairs

Defining Neighbors: Traveling Salesperson Problem

Approach: 2-change operation

- Select two edges to remove
- Flipping the edges between the removed edges
- Reconnect the tour to the flipped edges



Original tour: **A-B-C-D-E-F-G-H-A**

Select two edges to remove:

(B,C) and (E,F) A-**B** **C**-D-**E** **F**-G-H-A

Reverse the edges in-between:

C-D-E → E-D-C A-B **E-D-C** F-G-H-A

Reconnect the tour with the reversed elements:

A-**B-E**-D-**C-F**-G-H-A

Our new tour.

$f(s)$: Total weight of the path

Defining Neighbors: SAT

Example

(A ∨ ¬B ∨ C) ∧
(¬A ∨ C ∨ D) ∧
(B ∨ D ∨ ¬E) ∧
(¬C ∨ ¬D ∨ ¬E) ∧
(¬A ∨ ¬C ∨ E)

Approach: Flip one variable assignment

Example:

Starting state (A=T, B=F, C=T, D=T, E=T) $f(s) = 4$

Resulting Neighborhood:

(A=F, B=F, C=T, D=T, E=T) $f(s) = 4$

(A=T, B=T, C=T, D=T, E=T) $f(s) = 4$

(A=T, B=F, C=F, D=T, E=T) $f(s) = 5$

(A=T, B=F, C=T, D=F, E=T) $f(s) = 4$

(A=T, B=F, C=T, D=T, E=F) $f(s) = 4$

$f(s)$: Number of satisfied clauses

Hill Climbing: Tradeoffs

What is a neighbor?

Vaguely: for a given problem structure, neighbors are states that can be produced by a small change.

The Tradeoff!

Neighborhood too small?

Algorithm gets stuck in local optima

Neighborhood too large?

Algorithm is inefficient (too many neighbors to evaluate)

How to pick next state?

Greedy: Choose best neighbor

When to terminate?

When no neighbor has better value than current state

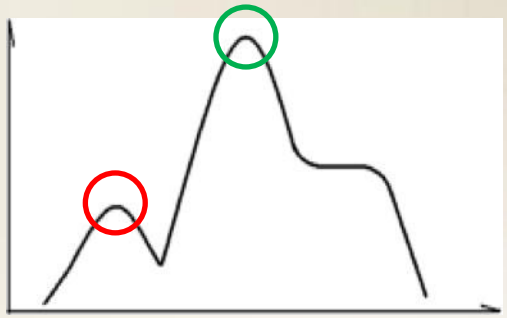
Hill Climbing Algorithm

```
Pick initial state s
Loop:
      Pick t ∈ neighbors(s) with the best f(t)
      If f(t) is not better than f(s) THEN break
      s ← t
return s
```

The algorithm stops when no neighbor is better

What could have happened? Local optima!

We may have found a "local optimum," not a "global optimum"



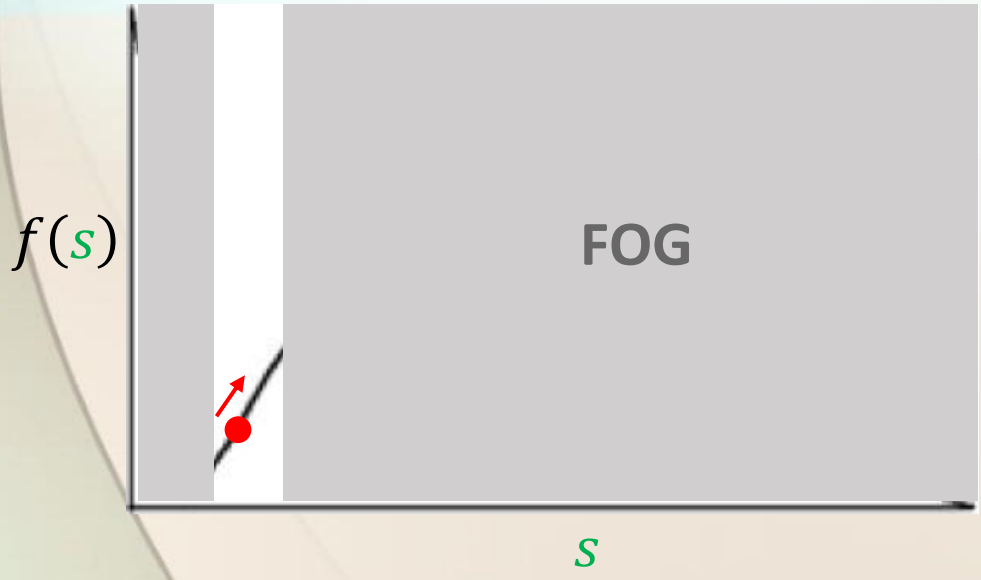
Hill Climbing: The Local Optima Problem

Our **space** is a complex landscape with multiple optima.

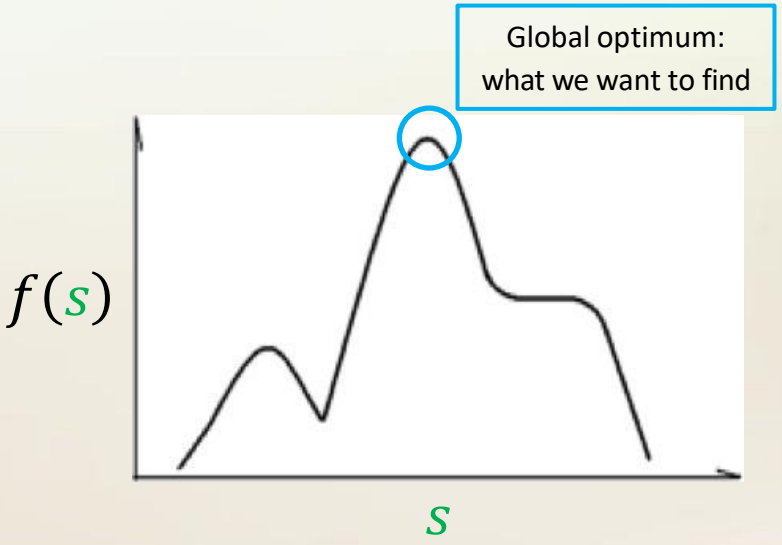
But we can only **see** locally: fog obscures everything except immediate neighbors

Our **goal** is to find the global optimum.

The **result** is that we climb to a local optimum and stop, even though a global optimum exists elsewhere.

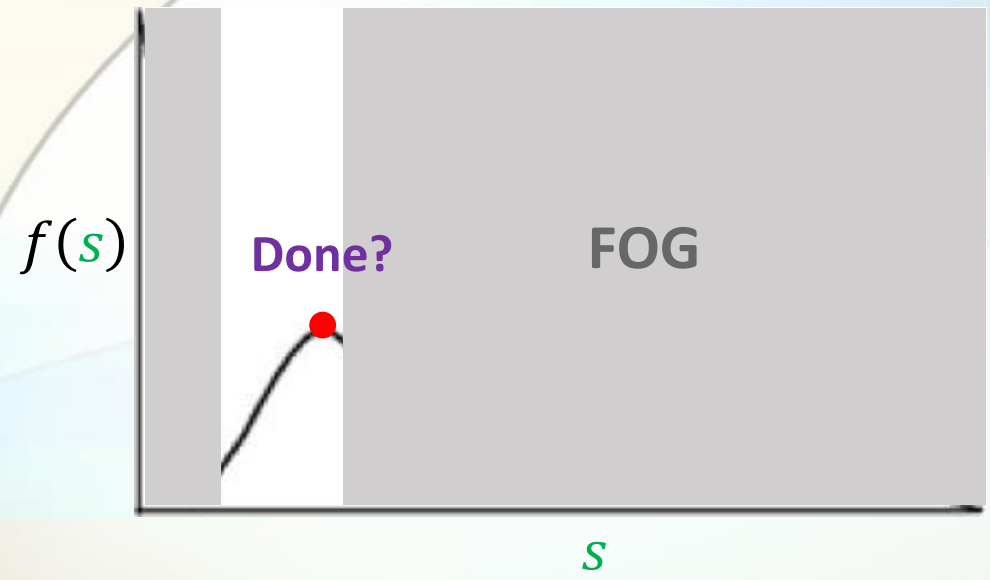


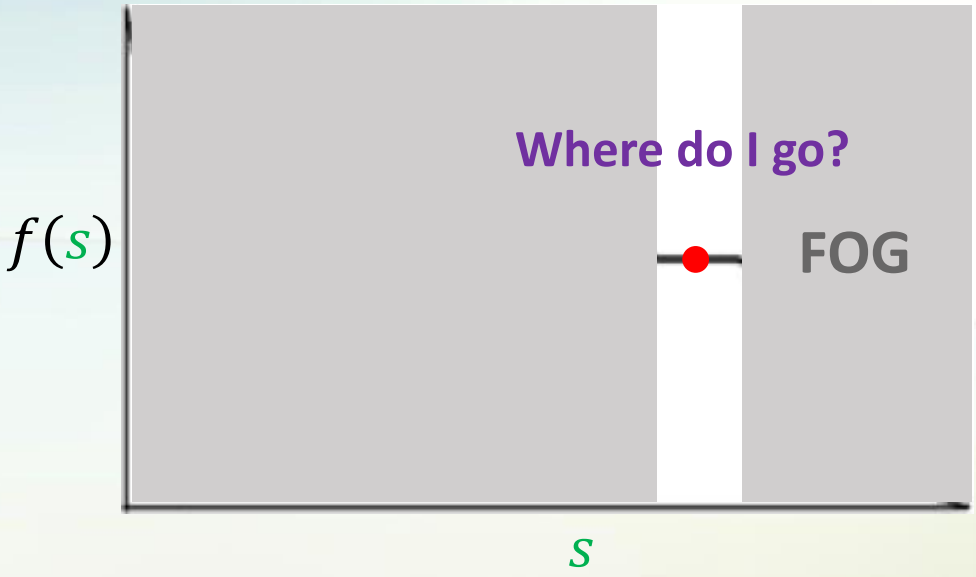
What we see



What is actually going on

Hill Climbing: Escaping Local Optima





How do we escape Local optima?

Strategy 1: Random Restarts

- When stuck, pick a random new starting point
- Run hill climbing again
- Repeat k times, return best result

Strategy 2: Stochastic Hill Climbing

- Transform $f(s)$ into a probability distribution over neighbors, with higher $f(s)$ values receiving higher probability
- Does not always pick the best neighbor

Questions

You are using hill climbing to solve a problem. The algorithm stops at a state where $f(s) = 15$, and all neighbors have $f(t) \leq 15$. Which of the following is TRUE?

A. You have definitely found the global optimum

✓ B. You have definitely found a local optimum, but it might also be the global optimum

C. The algorithm failed because it didn't improve the solution

D. You need to restart with a different objective function

E. All neighbors must have $f(t) = 15$ (no variation)

A hill climbing algorithm is solving a scheduling problem. After 50 iterations, it finds a solution with $f(s) = 87$. After 500 more iterations, we still have $f(s) = 87$. What is the BEST next step?

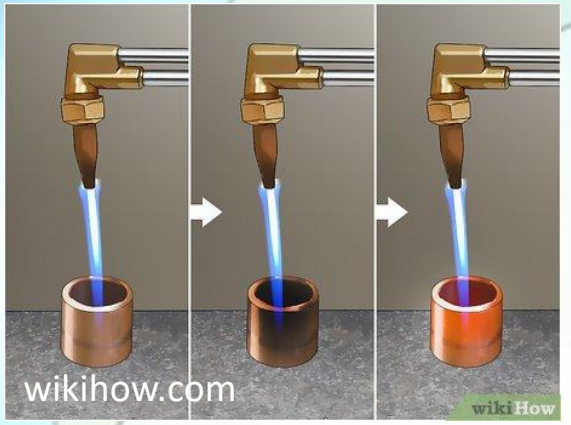
A. Declare the problem unsolvable and try a different algorithm

B. Continue running hill climbing longer—it just needs more time

✓ C. Use random restart: pick a new random initial state and run hill climbing again

D. Reduce the neighborhood size to find smaller improvements

E. Increase the neighborhood size to explore more options



Simulated Annealing

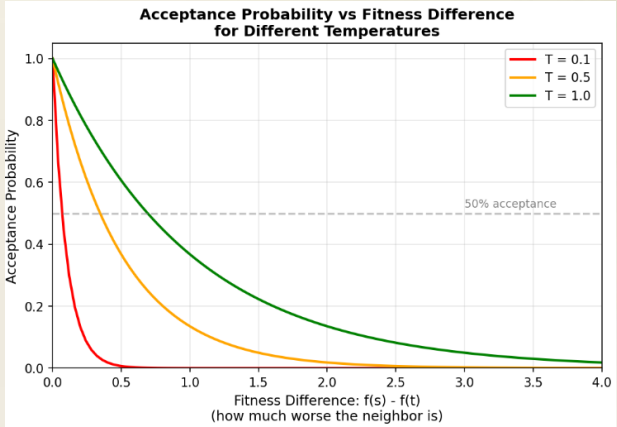
Inspiration: Annealing strengthens metal by heating it and cooling it slowly, relieving internal stresses and creating a uniform crystal structure.

Algorithmic analog: Allow "bad" moves early (high temperature) then cool by becoming increasingly selective over time; eventually behaving like hill climbing (low temperature).

```
s := choose initial state
best := s    # track best observed solution
T := 1       # temperature; this could be larger
K := 1       # iterations
for k = 0 through K:
    T := T · 0.99                # cool down
    t := neighbor(s)             # pick a random neighbor
    if f(t) > f(s), then
        s := t
        if f(t) > f(s), then best := t
    else with probability e^(-(|f(s)-f(t)|)/T) do s := t   # worse, but choose anyway
return best
```

The Probability Formula:

- Borrowed from physics (Boltzmann distribution in thermodynamics)
- Gives desired properties: gradual transition from ***exploration*** to ***exploitation***
- The exponential decay naturally handles the temperature cooling



Simulated Annealing: Temperature Cooling Schedule

How fast to decrease T?

Too fast:

Becomes hill climbing

Gets stuck in local optima

Too slow:

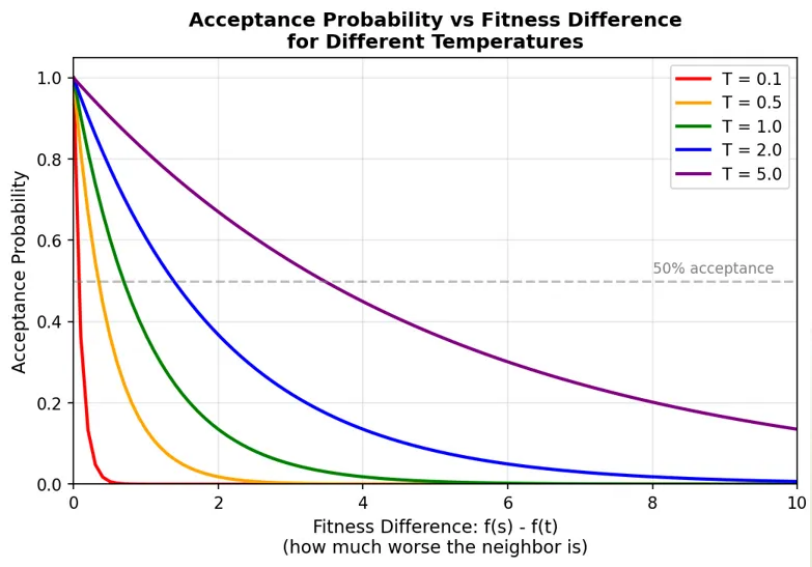
Takes too long to converge

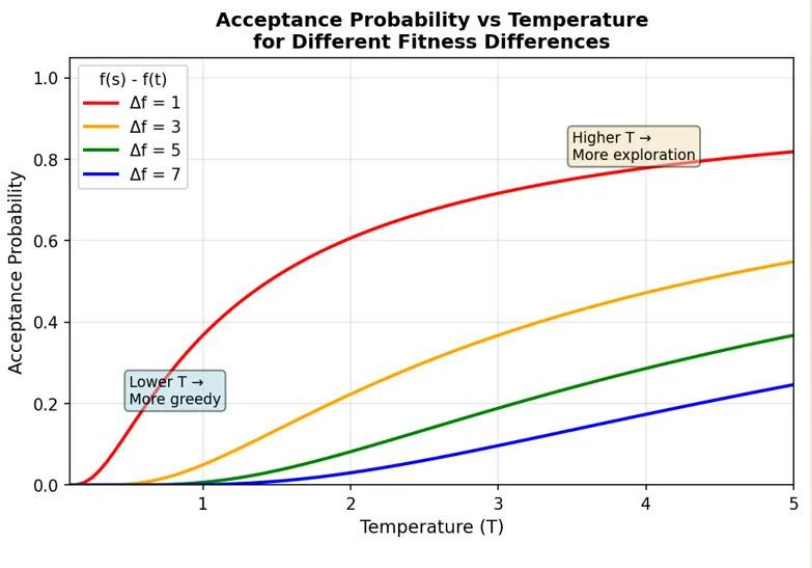
Wasteful exploration

Common approach:

Geometric decay or something similar

Commonly: $T_{t+1} = 0.99 \cdot T_t$





Evolution over time

Phase	Temperature	Acceptance of Worse Moves	Goal
Early	High	Frequent	Exploration
Middle	Moderate	Occasional	Refinement
Late	Low	Rare	Convergence

Question

Which of the following is likely to give the best cooling schedule for simulated annealing?

- ✗ *A.* $T_{t+1} = T_t \cdot 1.25$ — Temperate is increasing
- ✗ *B.* $T_{t+1} = T_t$ — Temperate is constant
- ✓ *C.* $T_{t+1} = T_t \cdot 0.8$
- ✗ *D.* $T_{t+1} = T_t \cdot 0.001$ — Cools way too fast; basically, hill-climbing

Questions

Distinction

- Use **A*** when: small branching factor, want the path, good heuristic exists
- Use **hill climbing** when: huge state space, only care about final solution, can define neighbors easily

Which of the following would be better to solve with hill climbing rather than A* search?

✓ i. Finding the smallest set of vertices such that every edge in the graph has at least one endpoint in the set

✓ ii. Finding the fastest way to schedule jobs with varying runtimes on machines with varying processing power

✗ iii. Finding the fastest way through a maze

Hill climbing: 2^n state space (n vertices), no path structure, unclear heuristic for A*;
Start with valid set, remove vertices to improve

Hill climbing: Large state space. only care about final schedule
Start random, swap jobs/machines to improve

For n jobs and m machines,
$n! \cdot m^n$ state space
$n!$: ordering of jobs on machines affects completion time
m^n : each job assigned to one of m machines

A*: Small branching factor, good heuristic (distance), need the path itself

Genetic Algorithms

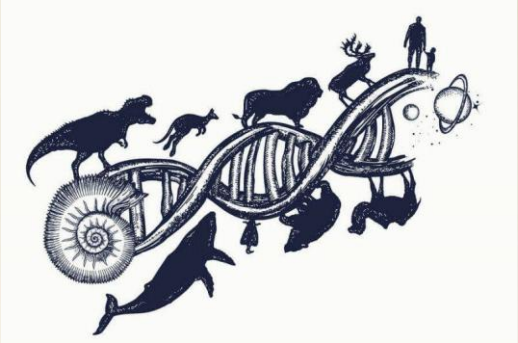
Genetic Algorithms

Inspiration: Biological Evolution

- Organisms encode genetic information in DNA
- Beneficial traits help survival and reproduction
- Population evolves toward better fitness over generations

Algorithmic Analog

- Encode solutions as “chromosomes”
- Better solutions have higher “fitness”
- Apply selection, crossover, and mutation operations



Genetic Algorithms: Biology ↔ Computer Science

Biology	CS Analog
DNA/Chromosome	String or array (state representation)
Gene	Single element in array (e.g., position `i`)
Population	Collection of candidate solutions
Fitness Function	Objective function value $f(s)$
Generation	Iteration of the algorithm: replace some or all of the old population with new offspring
Evolution	Iterative improvement over generations
Operations	
Natural Selection	Probabilistic selection based on $f(s)$: better fitness have higher probability of reproducing
Crossover/Recombination	Swap/merge portions of two arrays
Mutation	Randomly modify array element(s)

Genetic Algorithm Pseudocode

```
1. Let s₁, ..., s_N be the current population
2. Let p_i = f(s_i) / Σ_j f(s_j) be the reproduction probability
3. for k = 1; k < N; k += 2:
      parent1 = randomly pick according to p_i
      parent2 = randomly pick another
      Randomly select crossover point, swap substrings of
         parents 1, 2 to generate children t[k], t[k+1]
4. for k = 1; k ≤ N; k++:
     Randomly mutate each position in t[k] with small
       probability (mutation rate)
5. New generation replaces old: {s} ← {t}. Repeat.
```

Probability Distribution based on fitness of individuals; Higher fitness → higher chance of being selected.

Crossover to create a pair of children

Mutate: Possibly modify each gene (position) of each individual in the population

Repeat the process with the next generation

Example

Problem: Schedule 5 courses (A, B, C, D, E) into 3 time slots (Mon/Wed, Tue/Thu, Fri/Sat)

- Each course is assigned to exactly one time slot; multiple courses can share a slot
- Students can only attend one course per time slot

Goal: Maximize the number of students who can enroll in all their desired courses

Student Request	# Students
A, B, C	2
A, B, D	7
A, D, E	3
B, C, D	4
B, D, E	10
C, D, E	5

← 2 students want to take courses A, B, and C

Individual/Chromosome/State: Assignment of courses to time slots

Encoding: 5-character string (one character per course)

Character values: M (Mon/Wed), T (Tue/Thu), F (Fri/Sat)

Character positions: A = 1, B = 2, C = 3, D = 4, E = 5

Population: Collection of chromosomes

e.g. : { MMFTM, TTFMM, FMTTF, MTTTF }

Example chromosome: MMFTM

- A → Mon/Wed
- B → Mon/Wed
- C → Fri/Sat
- D → Tue/Thu
- E → Mon/Wed

Example: Fitness Calculation

What are we trying to optimize?

Maximize the number of students who can take all their desired courses.

What defines a valid solution for each student?

Their requested courses must be scheduled in different time slots (no conflicts).

How do we quantify solution quality?

Count satisfied constraints (students who can enroll without conflicts).

$f(chromosome)$ = # students who can enroll without conflicts

Example: Fitness Calculation

$f(chromosome)$ = students who can enroll without conflicts

$ABCDE$

$f(\text{MMFTM}) = 4 + 5 = 9$

Constraint (Student Request)	# Students	Time Slots	Satisfied?
A, B, C	2	**M**, **M**, F	No
A, B, D	7	**M**, **M**, T	No
A, D, E	3	**M**, T, **M**	No
B, C, D	4	**M**, F, T	**Yes**
B, D, E	10	**M**, T, **M**	No
C, D, E	5	F, T, **M**	**Yes**

$f(\text{MTFMF}) = 2 + 4 + 10 = 16$

Constraint (Student Request)	# Students	Time Slots	Satisfied?
A, B, C	2	**M**, T, F	**Yes**
A, B, D	7	**M**, T, **M**	No
A, D, E	3	**M**, **M**, F	No
B, C, D	4	T, F, **M**	**Yes**
B, D, E	10	T, **M**, F	**Yes**
C, D, E	5	F, **M**, F	No

Population and Parent Selection

For an initial population, we can randomly generate strings and calculate their fitness:

Individual: s	$f(s)$	$\sim\%$
MMFTM	9	26
TTFMM	4	11
FMTTF	19	54
MTTTF	3	9

$$\sum f(s) = 35$$

A ***parent*** is an individual from the current population that is selected to contribute genetic material to create offspring for the next generation.

Selecting Parents

Just like Stochastic Hill Climbing, we:

- Transform $f(s)$ into a probability distribution over individuals, with higher $f(s)$ values receiving higher probability
- Does not always pick the best individual

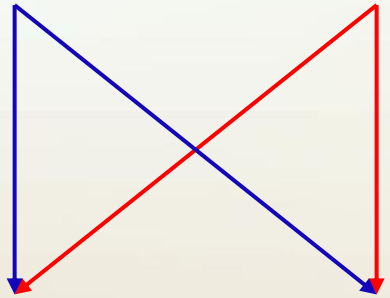
FMTTF is most likely to be selected as a parent but is not guaranteed.

Crossover

Crossover is the operation of combining genetic material from two parents to create ***two*** children.

This is the way for the genetic algorithm to **mix traits** from different solutions to explore new combinations.

Parents MMFTM FMTTF



Crossover Point: Index 2

Offspring MMTTF FMFTM

Mutation

Mutation is is the operation of randomly modifying individual genes (elements) of a chromosome.

This is a way for a genetic algorithm to introduce new genetic variation and explore solutions beyond the current population.

The **mutation rate** controls how often mutations occur (in each position of an individual).

Mutation Rate (μ)	Behavior	Result
0.001 (too low)	Rare mutations	Population lacks diversity, converges too quickly
0.05 (good)	Occasional mutations	Maintains diversity, steady improvement
0.5 (too high)	Frequent mutations	Good solutions disrupted, poor convergence

Original	MMFTM	FMTTF
	↓	↓
Mutated	MFFTM	FMFMM

Convergence: When individuals in the population become highly similar to each other and fitness values stabilize across generations.

The Next Generation

The Original Population

Individual: s	$f(s)$	$\sim\%$
MMFTM	9	26
TTFMM	4	11
FMTTF	19	54
MTTTF	3	9

$$\sum f(s) = 35$$

selection, crossover, mutation →

The Next Generation

Individual: s	$f(s)$	$\sim\%$
FMFTT	11	39
MMTTF	13	46
MMTFF	4	14
FTTTF	0	0

$$\sum f(s) = 28$$

Average fitness dropped: 8.75 → 7
Best fitness dropped: 19 → 13
Total fitness dropped: 35 → 28

We want to improve with each generation,
but our next generation is worse!

One solution: **Elitism**! Always keep the best individual(s) from the previous generation;
we can modify the algorithm as:

```
5. New generation replaces old: {s} ← {t}, except:

      Keep the best k individuals from {s} in {t} (elitism)
```

Genetic Algorithms: Key Parameters and Variations

Parameter / Choice	Options	Recommendation
Population size	20-200	Larger for complex problems
Mutation Rate	0.01-0.1	Start with 1/n (n = chromosome length)
Generations	100-10000	Run until fitness plateaus
Selection	Proportional / Rank / Tournament	Rank-based reduces extreme fitness effects
Crossover	Single-point / Multi-point / Uniform	Single-point is standard, simple
Elitism	Yes / No	always keep best solution(s)

Variation	Description	Benefit
Elitism	Keep best k individuals from previous generation	Never lose best solution found
Rank-based selection	Order raw fitness values and assign a rank (1, 2, 3, …)	Reduces influence of extreme fitness values
Multi-point crossover	Split at multiple positions instead of one	Increases genetic mixing
Adaptive mutation rate	Change μ over time (high → low)	More exploration early, exploitation later
Tournament selection	Pick best from random subset	Simpler than probability calculation

Choosing Variations

Elitism (Recommended): ensures best solution is never lost
Rank-based selection: Useful when fitness values vary widely
Adaptive parameters: requires tuning and experimentation

- Could also train a neural network to recommend parameters

Genetic Algorithms: Challenges

Challenge	Problem	Solution Approaches
Encoding design	How to represent solutions as chromosomes?	Design so crossover is meaningful; Ensure mutations maintain validity
Premature convergence	All individuals become identical early	Higher mutation rate; Use niching techniques (methods to maintain diversity by encouraging different subpopulations to explore different regions of the search space.
Parameter selection	Choosing N, μ, crossover rate (whether to apply crossover), etc.	Use rules of thumb ($\mu = 1/n$); Experiment; Adaptive parameters
Computational cost	Large populations × many generations	GPU parallelization; Adaptive stopping criteria: • No improvement stopping • Convergence threshold • Fitness target reached • Rate of improvement is too slow

In Summary

Comparing the Three Approaches

Approach	Local Optima?	Population?	Randomness?	Parameters	Best For
Hill Climbing	Vulnerable	Single state	Deterministic	Neighborhood definition	Simple problems, try first
Simulated Annealing	Better escape	Single state	Probabilistic	Temperature, cooling rate	When HC gets stuck
Genetic Algorithms	Best escape	Multiple states	Probabilistic	N, μ, selection, crossover	Hard, multimodal (many local optima) problems

Trade-offs

Hill climbing: Simplest, fastest per iteration, but gets stuck

Simulated annealing: More robust, requires tuning temperature

Genetic algorithms: Most complex, good for hard problems, many parameters

What We Saw

1. Optimization problems require different techniques than path-finding
2. Local search is efficient but can get trapped in local optima
3. Randomness helps escape local optima
4. Nature-inspired algorithms (annealing, evolution) provide powerful metaphors

Some Advice

- Start simple (hill climbing)
- Add sophistication as needed (simulated annealing, genetic algorithms)
- Experiment with parameters
- Consider problem structure when designing state encoding and neighborhoods